

Open Source LLMs

Complete Study Guide

Llama • Mistral • HuggingFace • Ollama • Fine-Tuning • LoRA

AI & Data Science Learning Journey
Phase 2 — AI Frameworks & Open Source Models

Topics: What are Open Source LLMs | Llama 2 & 3 | Mistral | Mixtral MoE | Quantization | HuggingFace | Ollama | Fine-Tuning | LoRA | RAG vs Fine-Tuning

Section 1: What Are Open Source LLMs?

Definition

An open source LLM (Large Language Model) is a model where the weights — the billions of numbers that define how the model thinks — are publicly released for anyone to download and use. This is in contrast to closed models like GPT-4 (OpenAI) or Claude (Anthropic) where the weights are kept private and you can only access them via an API.

Why Open Source LLMs Matter

- Data Privacy — your data never leaves your own machine or server
- Zero API Cost — no per-token charges, run unlimited queries for free
- Full Control — customize, fine-tune, and modify the model for your needs
- No Vendor Lock-in — not dependent on any company's pricing or availability
- Compliance — hospitals, banks, law firms can use AI without sending sensitive data externally

Closed Source LLMs	Open Source LLMs
GPT-4, Claude, Gemini	Llama 3, Mistral, Mixtral, Phi-3, Gemma
Weights are private	Weights are publicly available
Must call via API	Download and run locally
Pay per token/call	Free to run (only hardware cost)
Data sent to their servers	Data stays on your machine
Cannot modify the model	Can fine-tune on your own data

Section 2: Key Models — Llama & Mistral

2.1 — Llama 2 and Llama 3 (by Meta)

Llama (Large Language Model Meta AI) was released by Meta (Facebook) and shocked the AI world by being as capable as closed models but completely open. Llama 2 was released in 2023 and Llama 3 in 2024 — a significant upgrade in quality.

Model	Size Options	Key Facts
Llama 2	7B, 13B, 70B	Released 2023. Good baseline. Llama 2 Chat is fine-tuned for conversation.
Llama 3	8B, 70B	Released 2024. Much better than Llama 2. Stronger reasoning and coding.
Llama 3.1	8B, 70B, 405B	Released mid-2024. 405B rivals GPT-4. Best open source model available.
Llama 3.2	1B, 3B, 11B, 90B	Includes multimodal (vision) versions. Very efficient small models.

Key Rule	Always prefer Llama 3 over Llama 2 for new projects. Llama 3 is significantly smarter, better at following instructions, and supports a larger context window.
----------	--

2.2 — Mistral 7B (by Mistral AI)

Mistral AI is a French startup that released Mistral 7B in September 2023. It immediately made headlines by outperforming Llama 2 13B — a model nearly TWICE its size. This showed that model architecture matters as much as raw size.

- Uses Sliding Window Attention — efficiently handles longer contexts
- Uses Grouped Query Attention — faster inference with less memory
- Outperforms Llama 2 13B despite being nearly half the size
- Extremely popular for production use — great quality-to-speed ratio

2.3 — Mixtral 8x7B — Mixture of Experts

Mixtral 8x7B is Mistral AI's most powerful model, using a revolutionary architecture called Mixture of Experts (MoE). Understanding this architecture is important for interviews.

How Mixture of Experts Works

A normal 7B model activates ALL its parameters for every single token it processes. Mixtral takes a different approach:

- Mixtral has 8 'expert' networks, each with 7B parameters
- A 'router' network decides which 2 experts are most relevant for each token
- Only those 2 experts activate — the other 6 stay dormant
- Total parameters: 47B, but active parameters per token: only ~13B

Input Token --> Router --> Expert 3 + Expert 7 activated --> Output
(Expert 1, 2, 4, 5, 6, 8 stay dormant for this token)

Result You get the knowledge capacity of a 47B model but only pay the compute cost of a ~13B model. This is why Mixtral is so efficient for production use.

2.4 — Other Popular Open Source Models

Model	By	Best For
Phi-3 Mini (3.8B)	Microsoft	Runs on very low RAM. Surprisingly capable for its tiny size.
Phi-3 Medium (14B)	Microsoft	Strong reasoning. Great for coding and math tasks.
Gemma 2 (2B, 9B, 27B)	Google	Fast, efficient. Good for general purpose tasks.
CodeLlama (7B-34B)	Meta	Specialized for writing, explaining, and reviewing code.
DeepSeek Coder	DeepSeek	Top performer for coding tasks. Strong alternative to CodeLlama.
Qwen 2.5	Alibaba	Strong multilingual support including Chinese and Hindi.

Section 3: Model Sizes and Quantization

3.1 — Understanding Model Sizes

LLM sizes are measured in parameters — the billions of learned numbers that define the model's knowledge and capabilities. More parameters generally means smarter but requires more memory and is slower.

Size	Parameters	Hardware Needed
Small	1B - 3B	Any laptop, even 4GB RAM — great for learning
Medium-Small	7B - 8B	8GB RAM laptop — practical for most use cases
Medium	13B - 14B	16GB RAM laptop — good quality, still runs locally
Large	70B	GPU server or high-end workstation (64GB+ RAM)
XL	405B	Multiple high-end GPUs — enterprise grade

3.2 — Quantization

A 7B model in full precision (FP32) would need ~28GB of RAM — too much for most laptops. Quantization solves this by reducing the numerical precision of the model's weights, dramatically shrinking memory requirements with minimal quality loss.

Think of it like image compression: a high-resolution photo takes 10MB, a compressed JPEG takes 1MB and looks almost identical. Quantization does the same for model weights.

Format	Memory for 7B Model	Quality
FP32 (full precision)	~28 GB	Reference quality — rarely used
FP16 / BF16	~14 GB	Excellent — standard for GPU inference
Q8 (8-bit)	~8 GB	Very good — barely noticeable quality drop
Q4 (4-bit)	~4 GB	Good — default for Ollama

		downloads
Q2 (2-bit)	~2 GB	Acceptable — noticeable quality drop

Ollama Default	When you run 'ollama pull llama3', Ollama automatically downloads the Q4 quantized version. This gives you a good balance of quality and memory usage for local machines.	
-----------------------	---	--

Section 4: HuggingFace — The GitHub of AI

What is HuggingFace?

HuggingFace (huggingface.co) is the world's largest repository and community for open source AI models. Just as GitHub is where developers store and share code, HuggingFace is where AI researchers store and share trained models.

GitHub	HuggingFace
Stores code (repositories)	Stores AI models (model hub)
Pull requests and issues	Model cards and datasets
npm / pip packages	Transformers / PEFT libraries
GitHub Actions (CI/CD)	Spaces (live model demos)
500M+ repositories	500K+ models

Key Components of HuggingFace

- Model Hub — browse and download 500,000+ pre-trained models for any task
- Datasets — 50,000+ public datasets for training and evaluation
- Spaces — interactive demos where you can try models directly in the browser
- Inference API — run models without downloading them (similar to OpenAI API)

Key HuggingFace Python Libraries

Library	Purpose
transformers	Load, run, and fine-tune any model. The core library.
datasets	Load public datasets for training with one line of code
peft	Parameter Efficient Fine-Tuning — LoRA, QLoRA, adapters
trl	Training with Reinforcement Learning from Human Feedback (RLHF)
accelerate	Distribute model training across multiple GPUs seamlessly
bitsandbytes	Quantization library — enables 4-bit and 8-bit model loading
tokenizers	Fast tokenizers for processing text before feeding to model
evaluate	Metrics and evaluation tools for measuring model performance

Loading a Model from HuggingFace — Code

```
from transformers import AutoTokenizer, AutoModelForCausalLM
import torch

# Load tokenizer and model from HuggingFace Hub
model_name = 'microsoft/phi-2' # small 2.7B model, good for learning

tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(
    model_name,
    torch_dtype=torch.float16, # FP16 to save memory
    device_map='auto'         # auto-detect GPU or CPU
)

# Generate text
inputs = tokenizer('Explain what RAG means in AI:', return_tensors='pt')
outputs = model.generate(**inputs, max_new_tokens=100)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

Using the Pipeline API (Easiest Way)

```
from transformers import pipeline

# One-liner to load and run any model
generator = pipeline('text-generation', model='gpt2')
result = generator('The future of AI is', max_length=50)
print(result[0]['generated_text'])

# Other pipeline tasks:
# 'text-classification' - sentiment analysis
# 'question-answering' - QA from context
# 'summarization' - summarize long text
# 'translation' - translate between languages
# 'fill-mask' - fill in missing words
```

Section 5: Ollama — Run LLMs Locally

What is Ollama?

Ollama is the simplest way to run open source LLMs on your own machine. It handles everything: downloading the model, quantization, running a local server, and managing multiple models. Think of it as Docker — but for AI models.

Analogy

Docker manages containers for applications. Ollama manages models for AI. Both give you a simple CLI to pull, run, and manage complex software.

Installation

- Go to <https://ollama.com>
- Download for your OS (Windows, Mac, Linux)
- Install — it starts a background service on port 11434

Ollama CLI Commands

```
# Download and start chatting with a model
ollama run llama3

# Just download without starting a chat
ollama pull mistral

# List all downloaded models
ollama list

# Remove a model to free disk space
ollama rm mistral

# See model details
ollama show llama3

# Run model with custom system prompt
ollama run llama3 'You are a Python expert. Help me with code.'
```

Using Ollama in Python with LangChain

```
from langchain_ollama import ChatOllama
```

```
# Connect to local Ollama server
llm = ChatOllama(model='llama3', temperature=0.3)

# Use exactly like any other LangChain LLM
response = llm.invoke('Explain what RAG is in simple terms')
print(response.content)
```

Swap Cloud LLM to Local — One Line Change

```
# BEFORE: Using Groq (cloud, free but external)
from langchain_groq import ChatGroq
llm = ChatGroq(model='llama3-8b-8192', temperature=0.3)

# AFTER: Using Ollama (fully local, private, free)
from langchain_ollama import ChatOllama
llm = ChatOllama(model='llama3', temperature=0.3)

# Everything else in your app stays EXACTLY the same!
# CrewAI agents, LangChain chains, LlamaIndex — all work unchanged.
```

Popular Models for Ollama

Command	Model
ollama run llama3	Meta Llama 3 8B — general purpose, most popular
ollama run mistral	Mistral 7B — efficient, great quality
ollama run mixtral	Mixtral 8x7B — powerful, uses MoE architecture
ollama run phi3	Microsoft Phi-3 Mini — tiny but capable, low RAM
ollama run codellama	Meta CodeLlama — specialized for coding tasks
ollama run gemma2	Google Gemma 2 — fast and efficient

Section 6: Fine-Tuning Open Source LLMs

What is Fine-Tuning?

A base model like Llama 3 is trained on the entire internet — it knows everything generally but nothing specifically about your domain. Fine-tuning takes a pretrained model and trains it further on YOUR specific data to create an expert in your domain.

Base Llama 3	Fine-Tuned Llama 3
Knows everything generally	Expert in medical diagnosis
Generic tone and style	Speaks like your company brand
All languages equally	Focused on Hindi + English

Real-World Fine-Tuning Examples

- CodeLlama = Llama 2 fine-tuned on millions of lines of code
- Medical chatbot = Llama 3 fine-tuned on clinical notes and medical textbooks
- Customer support bot = Mistral fine-tuned on your company's support tickets
- Legal assistant = Llama 3 fine-tuned on case law and legal documents

6.1 — Types of Fine-Tuning

Type	What Changes	Cost
Full Fine-Tuning	ALL model parameters are retrained	Very expensive — needs many GPUs, days of training
LoRA	Only small adapter matrices are trained	Cheap — runs on a single GPU in hours
QLoRA	LoRA + 4-bit quantization	Very cheap — runs on a consumer GPU (RTX 3090)
PEFT	Umbrella term for LoRA, QLoRA, prefix tuning etc.	Varies, but all are more efficient than full fine-tuning

6.2 — LoRA Explained

LoRA (Low-Rank Adaptation) is the most popular fine-tuning method today. Instead of updating all 7 billion parameters, LoRA adds two small matrices (adapters) to certain layers and only trains those.

Original Weight Matrix (7B params, FROZEN)
+
LoRA Matrix A (tiny, TRAINABLE) x LoRA Matrix B (tiny, TRAINABLE)
Only 0.1% to 1% of parameters are trained — massive savings!

QLoRA — Fine-Tuning on Consumer Hardware

QLoRA combines LoRA with 4-bit quantization. The base model is loaded in 4-bit (very small) and only the LoRA adapters are trained in full precision. This means you can fine-tune a 7B model on a single RTX 3090 GPU with 24GB VRAM.

```
# QLoRA fine-tuning setup (conceptual example)
from transformers import AutoModelForCausalLM, BitsAndBytesConfig
from peft import LoraConfig, get_peft_model

# Step 1: Load base model in 4-bit
bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_compute_dtype=torch.float16
)
model = AutoModelForCausalLM.from_pretrained('meta-llama/Llama-3-8B',
    quantization_config=bnb_config)

# Step 2: Add LoRA adapters
lora_config = LoraConfig(
    r=16,                # rank of adapter matrices (higher = more params)
    lora_alpha=32,        # scaling factor
    target_modules=['q_proj', 'v_proj'], # which layers to adapt
    lora_dropout=0.05
)
model = get_peft_model(model, lora_config)

# Only 0.5% of parameters are now trainable!
model.print_trainable_parameters()
# Output: trainable params: 4,194,304 || all params: 8,030,261,248 (0.05%)
```

Section 7: RAG vs Fine-Tuning — When to Use Which

This is one of the most common AI architecture questions in interviews. Both RAG and fine-tuning can improve an LLM's performance, but they solve different problems.

	RAG	Fine-Tuning
What it does	Retrieves relevant documents at query time and gives them to the LLM as context	Trains the model on new data, baking knowledge directly into its weights
When to use	Knowledge changes frequently (news, docs, databases)	Style, tone, or domain expertise needs to be persistent
Example use case	Q&A over your company's latest documentation	Customer support bot that always responds in your brand voice
Cost	Low — just a vector database and retriever	High — GPU time for training
Update process	Just add/update documents in the vector store	Must retrain and re-deploy the model
Data freshness	Always up to date — query fresh docs at runtime	Stale — knowledge frozen at training time
Hallucination risk	Lower — answers grounded in retrieved documents	Higher — model generates from memory

Best Practice	In production, most companies use BOTH: fine-tune for personality/style/domain, and RAG for factual knowledge. For example: fine-tune a medical LLM on clinical terminology, then use RAG to answer questions from the latest research papers.
----------------------	--

Section 8: The Full Ecosystem — Big Picture

Every technology you have learned connects together into a complete AI application stack. Understanding this big picture is essential for system design discussions.

HuggingFace Hub	Source of open source models — download Llama 3, Mistral, etc.
Ollama / Transformers	Run models locally — handles quantization, inference, serving
LangChain / LlamaIndex	Framework layer — chains, RAG, document loading, vector stores
CrewAI	Multi-agent layer — define agent crews that use LLMs to collaborate
FastAPI	API layer — expose your AI system as REST endpoints
AWS / Azure	Cloud deployment — scale, host, and secure your application

Section 9: Interview Questions & Answers

Review these Q&A cards to reinforce your understanding and prepare for technical discussions.

Q: What is an open source LLM and why does it matter for enterprise AI?

A: An open source LLM is a model whose weights are publicly released — anyone can download and run it. It matters for enterprise because: (1) Data privacy — sensitive data never leaves your infrastructure. (2) Zero API cost — no per-token charges. (3) Full control — customize and fine-tune for your domain. (4) Compliance — regulated industries (healthcare, banking, law) can use AI without sending data to third parties.

Q: What is the difference between Llama 2 and Llama 3?

A: Both are open source LLMs by Meta. Llama 2 (2023) was the first major open release and comes in 7B, 13B, 70B sizes. Llama 3 (2024) is a significant upgrade — better reasoning, stronger instruction following, larger context window (8K vs 4K tokens), and improved coding ability. For any new project today, always use Llama 3. Llama 2 is mostly kept as a reference.

Q: What is Mixture of Experts (MoE) and how does Mixtral use it?

A: Mixture of Experts is an architecture where the model contains multiple 'expert' sub-networks. For each input token, a router network selects only 2-3 experts to activate, while the rest stay dormant. Mixtral 8x7B has 8 expert networks of 7B parameters each. Only 2 activate per token, so you get ~47B total capacity but only ~13B active compute. Result: quality of a large model at the cost of a smaller model.

Q: What is quantization and why is it important for running LLMs locally?

A: Quantization reduces the numerical precision of model weights — for example, from 32-bit floats (FP32) to 4-bit integers (Q4). A 7B model in FP32 needs ~28GB RAM, but in Q4 it only needs ~4GB — small enough to run on a laptop. The quality loss is minimal for Q4 and Q8. Ollama automatically uses Q4 quantization when you download a model, making local LLM inference accessible to everyone.

Q: What is HuggingFace and what are its key libraries?

A: HuggingFace is the largest repository of open source AI models — like GitHub but for AI. Key libraries: transformers (load/run any model), datasets (public training data), peft (LoRA/QLoRA fine-tuning), trl (RLHF training), accelerate (multi-GPU training), bitsandbytes (quantization), evaluate (model metrics). The transformers pipeline API lets you run any model in 3 lines of code.

Q: What is Ollama and how does it simplify local LLM usage?

A: Ollama is a tool that makes running LLMs locally as simple as running Docker containers. Commands like 'ollama run llama3' automatically download, quantize, and serve the model on localhost:11434. It integrates with LangChain via ChatOllama, meaning you can swap OpenAI for a local model with just one line change. No API keys, no costs, no data leaving your machine.

Q: What is fine-tuning and when would you use it over RAG?

A: Fine-tuning retrains a model on domain-specific data, baking knowledge into the weights. Use fine-tuning when: you need consistent style/tone, domain-specific terminology, or task-specific behavior that doesn't change often. Use RAG when: knowledge changes frequently, you need up-to-date information, or you want to reduce hallucinations by grounding answers in documents. Best practice: use both — fine-tune for personality/domain, RAG for factual knowledge.

Q: What is LoRA and why is it preferred over full fine-tuning?

A: LoRA (Low-Rank Adaptation) fine-tunes a model by adding small trainable adapter matrices to certain layers instead of updating all parameters. Only 0.1-1% of parameters are trained, reducing GPU memory by 3-5x and training time by 10x. A 7B model that needs 8 GPUs for full fine-tuning can be LoRA fine-tuned on a single GPU. The fine-tuned adapters can be saved separately and loaded on top of any base model.

Q: What is QLoRA and how does it differ from LoRA?

A: QLoRA combines LoRA with 4-bit quantization. The base model is loaded in 4-bit (very compressed), and only the small LoRA adapters are trained in full precision. This makes fine-tuning accessible on consumer hardware — you can fine-tune a 7B model on a single RTX 3090 GPU with 24GB VRAM. QLoRA from the PEFT library by HuggingFace is the standard method for fine-tuning on limited hardware.

Q: How would you swap a cloud LLM for a local one in a LangChain application?

A: It requires only one line change. Replace: `from langchain_groq import ChatGroq / llm = ChatGroq(model='llama3-8b-8192')` with: `from langchain_ollama import ChatOllama / llm = ChatOllama(model='llama3')`. Everything downstream — chains, agents, CrewAI crews, LlamaIndex query engines — works identically because they all accept a standard LangChain LLM object. This is the power of LangChain's abstraction layer.

Q: Name 5 popular open source LLMs and their key characteristics.

A: 1) Llama 3 (Meta) — best general-purpose open source model, 8B and 70B sizes. 2) Mistral 7B (Mistral AI) — extremely efficient, beats larger Llama 2 models. 3) Mixtral 8x7B (Mistral AI) — Mixture of Experts, powerful for complex tasks. 4) Phi-3 Mini (Microsoft) — tiny

3.8B model, runs on very low RAM, surprisingly capable. 5) CodeLlama (Meta) — specialized for coding, based on Llama 2, great for developer tools.

Section 10: Quick Summary Cheat Sheet

Term	One-Line Summary
Open Source LLM	Model with publicly released weights — download, run, fine-tune for free
Llama 3	Meta's best open source LLM — 8B and 70B sizes
Mistral 7B	Efficient 7B model that beats larger competitors — great quality/speed ratio
Mixtral 8x7B	MoE model — 8 experts, only 2 activate per token — efficient and powerful
Quantization	Reduce model precision (FP16 → Q4) to shrink memory — Q4 is 3.5x smaller than FP16
HuggingFace	GitHub for AI models — 500K+ models, transformers library, datasets
Ollama	Run any LLM locally with one command — handles download, quantization, serving
Fine-Tuning	Continue training a model on your data to make it a domain expert
LoRA	Fine-tune only tiny adapter matrices — 0.1% of params, runs on 1 GPU
QLoRA	LoRA + 4-bit quantization — fine-tune 7B models on consumer hardware
PEFT	HuggingFace library for LoRA/QLoRA — Parameter Efficient Fine-Tuning
RAG vs Fine-Tuning	RAG for dynamic knowledge, fine-tuning for style/domain expertise
Mixture of Experts	Multiple specialist networks — router picks best 2 per token
ChatOllama	LangChain class to connect to local Ollama models — drop-in for ChatOpenAI